

Deep Learning: Comprehensive Theory and Concepts Study Guide

Module 1: Deep Learning Basics

1. Fundamentals

- **What is Deep Learning?** Deep Learning is a subfield of Machine Learning based on Artificial Neural Networks (ANNs) containing multiple hidden layers (Deep Neural Networks). It automatically extracts hierarchical representations from raw input data.
- **AI vs ML vs Deep Learning**
 - **AI:** Simulating human cognitive processes using machines.
 - **ML:** Statistical algorithms learning patterns from historical data.
 - **Deep Learning:** Subset of ML that uses multi-layer neural networks to automatically learn features.
- **Why Deep Learning?** Traditional ML algorithms reach a performance ceiling (plateau) even as the volume of training data grows. Deep Learning models continue to improve in accuracy as they are fed larger volumes of data.
- **Applications of Deep Learning**
 - Computer Vision: Object detection, autonomous driving, medical imaging.
 - Natural Language Processing: Translation, sentiment analysis, generative chat.
 - Generative AI: Large Language Models, deepfakes, image synthesis.
 - Speech Recognition: Virtual assistants (Siri, Alexa).
- **Advantages of Deep Learning**
 - **Automatic Feature Extraction:** Eliminates the need for manual feature engineering.
 - **Unstructured Data Processing:** Excels at processing raw images, audio, video, and text.
 - **Excellent Scalability:** Performance scales effectively with massive datasets and GPU computing.
- **Disadvantages of Deep Learning**
 - **Data-Hungry:** Requires massive volumes of labeled training data.
 - **High Compute Cost:** Requires specialized hardware (GPUs/TPUs) to train.
 - **Lack of Interpretability:** Behave as "Black Boxes," making it hard to explain predictions.
 - **Prone to Overfitting:** Quickly overfits on small datasets if not regularized.

- **When should you use Deep Learning instead of Machine Learning?** Use Deep Learning when you have high volumes of unstructured data (images, text, audio) and sufficient GPU resources. Prefer traditional ML for structured tabular data, low data volume situations, or when model interpretability is a hard requirement.
 - **Deep Learning Workflow** [Data Prep] → [Define Network Architecture] → [Forward Pass] → [Loss Calculation] → [Backpropagation] → [Weight Update (Optimizer)] → [Evaluation/Tuning]
 - **Types of Neural Networks**
 - **ANN (Artificial Neural Networks):** Fully connected feed-forward networks for tabular data.
 - **CNN (Convolutional Neural Networks):** Specialized for spatial grid data (images).
 - **RNN (Recurrent Neural Networks):** Specialized for sequential/time-series data.
 - **Transformers:** Attention-based networks dominating modern sequence modeling and LLMs.
 - **GANs (Generative Adversarial Networks):** Generative networks for synthesizing new data.
 - **History of Deep Learning** Starts with the McCulloch-Pitts neuron (1943) and Rosenblatt's Perceptron (1958). Backpropagation was popularized in 1986. The modern era began in 2012 when **AlexNet** won the ImageNet challenge by a large margin, demonstrating the power of deep CNNs accelerated by GPUs. The Transformer architecture (2017) revolutionized NLP.
-

Module 2: Artificial Neural Networks (ANN)

- **What is ANN?** A computing system inspired by biological neural networks, consisting of layers of interconnected processing units called artificial neurons (nodes).
- **Biological Neuron vs Artificial Neuron**
 - **Dendrites** → **Inputs (X_i):** Receives signals from other cells.
 - **Cell Body (Soma)** → **Summation & Activation ($\sum w_i x_i + b$):** Combines inputs and fires when a threshold is met.
 - **Axon** → **Output (y):** Conducts electrical impulses to other neurons.
 - **Synapse** → **Weights (W_i):** Controls the strength of connection between neurons.
- **Components of ANN**
 - **Input Layer:** Receives raw features directly. No mathematical operations occur here.
 - **Hidden Layer:** Intermediary layers that perform matrix calculations and extract feature representations.
 - **Output Layer:** Produces the final model prediction (e.g. class probabilities, regression value).
 - **Weights (W):** Parameters representing connection strengths between neurons.

- **Bias (b):** An offset parameter added to the weighted inputs, allowing the activation function to shift left or right.
 - **Activation Function:** Applies non-linear transformations to node outputs, enabling the network to learn non-linear boundaries.
 - **Forward Propagation:** The process of passing inputs through the network layers to calculate the final output.
-

Module 3: Perceptron

- **What is Perceptron?** The simplest single-layer neural network model introduced by Frank Rosenblatt. It is a linear binary classifier:

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

- **Single Layer Perceptron:** Contains only input and output nodes. It can only draw a straight linear decision boundary.
 - **Multi-Layer Perceptron (MLP):** Integrates one or more hidden layers with non-linear activation functions, allowing it to approximate non-linear decision boundaries.
 - **Limitations of Perceptron**
 - Cannot classify non-linearly separable data.
 - **XOR Problem** A classic demonstration of perceptron limitations. A single-layer perceptron cannot solve the XOR logic gate because the classes cannot be separated by a single straight line.
 - **Why MLP?** Adding a hidden layer with non-linear activations resolves the XOR problem by projecting the input space into a higher-dimensional space where classes become linearly separable (Universal Approximation Theorem).
-

Module 4: Activation Functions

- **What is Activation Function?** A mathematical function applied to the output of a neuron.
- **Why Activation Function?** Without non-linear activation functions, any number of hidden layers in a neural network collapses mathematically into a single linear regression model. Non-linearity is essential to learn complex patterns.
- **Common Activation Functions**
 - **Sigmoid:** Maps inputs to range $(0, 1)$. Formula: $f(x) = \frac{1}{1+e^{-x}}$. Used for binary classification output layers.
 - *Cons:* Saturation causes **Vanishing Gradients**; outputs are not zero-centered.

- **Tanh (Hyperbolic Tangent):** Maps inputs to range $(-1, 1)$. Formula: $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$. Zero-centered.
 - *Cons:* Subject to vanishing gradients.
 - **ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$. Standard activation for hidden layers.
 - *Pros:* Fast to calculate; does not saturate for positive inputs, reducing vanishing gradients.
 - *Cons:* **Dying ReLU** (neurons outputting exactly 0 for negative inputs become permanently inactive/dead).
 - **Leaky ReLU:** $f(x) = \max(\alpha x, x)$ where $\alpha \approx 0.01$. Prevents the dying ReLU problem.
 - **ELU (Exponential Linear Unit):** Smoothly handles negative inputs, reducing bias shift.
 - **Softmax:** Formula: $f(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$. Maps outputs to a probability distribution summing to 1. Used in multi-class classification output layers.
 - **Swish:** Google's activation $f(x) = x \cdot \text{sigmoid}(\beta x)$. Smooth and non-monotonic, outperforming ReLU in deep networks.
 - **GELU (Gaussian Error Linear Unit):** Weights inputs by their value probability. Used in BERT and GPT architectures.
 - **Comparison**
 - *ReLU vs Sigmoid:* ReLU is faster to calculate and prevents vanishing gradients. Sigmoid saturates at boundaries, leading to flat gradients.
 - *Softmax vs Sigmoid:* Sigmoid is used for binary or multi-label classification (independent probabilities). Softmax is used for mutually exclusive multi-class classification (sum of probabilities = 1.0).
-

Module 5: Forward & Backpropagation

- **Forward Propagation** The process where inputs travel forward through the network layers. Each layer calculates $z = Wx + b$, applies the activation function $a = \sigma(z)$, and passes the output to the next layer until the output layer calculates the loss.
- **Backpropagation** The algorithm used to calculate the gradient of the loss function with respect to weights and biases. It operates backward from the output layer to the input layer.
- **Chain Rule** The mathematical foundation of backpropagation. It calculates the derivative of a composite function:

$$\frac{\partial \text{Loss}}{\partial W} = \frac{\partial \text{Loss}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial W}$$

- **Gradient Calculation & Weight Update** Once gradients are calculated, weights are updated using an optimizer and learning rate:

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial \text{Loss}}{\partial W}$$

- **Why Backpropagation?** It is computationally efficient ($O(N)$ space/time) because it reuses partial derivative values computed at output layers during backward passes, avoiding redundant calculations.
 - **Computational Graph** A directed graph where nodes represent variables and operations. Frameworks like PyTorch and TensorFlow construct computational graphs at runtime to automate gradient calculation (Autograd).
-

Module 6: Gradient Descent

- **Gradient Descent:** Optimization algorithm that iteratively updates weights in the direction of the negative gradient to find the local minimum of the loss function.
 - **Variations**
 - **Batch Gradient Descent:** Computes gradients across the entire dataset before updating weights. Stable but slow and memory-intensive.
 - **Stochastic Gradient Descent (SGD):** Updates weights after processing each individual sample. Fast but results in a noisy, oscillating optimization path.
 - **Mini-Batch Gradient Descent:** Updates weights after processing a small batch of samples (e.g., 32, 64). Standard training configuration.
 - **Learning Rate** The step size taken toward the minimum. Too high: model diverges and overshoots. Too low: model takes too long or gets stuck in local minima.
 - **Learning Rate Scheduling:** Decaying the learning rate over epochs (e.g. Cosine Annealing, Step Decay) to help the model settle into local minima.
 - **Momentum:** Accumulates past gradients to speed up updates in the correct direction, dampening oscillations.
 - **Nesterov Momentum:** Computes the gradient at an estimated "look-ahead" position, improving convergence speeds.
-

Module 7: Optimizers

- **SGD:** Basic weight update. Slow to converge on saddle points.
- **Momentum:** Accelerates SGD in consistent directions.
- **RMSProp:** Adapts the learning rate by dividing gradients by a running average of their recent magnitudes, resolving oscillations.
- **Adam (Adaptive Moment Estimation):** Combines the principles of Momentum (first moment of gradients) and RMSProp (second moment of gradients). Highly robust default optimizer.
- **AdamW:** A variation of Adam that decouples weight decay (L2 regularization) from the gradient update calculations, improving model generalization.

- **AdaGrad**: Shrinks learning rates for frequently updated parameters, useful for sparse feature datasets.
 - **AdaDelta**: Improves AdaGrad by limiting the accumulation window of past gradients to prevent learning rates from shrinking too fast.
 - **Which Optimizer is Best?**
 - **Adam / AdamW** is the standard default for most deep learning architectures (Transformers, deep MLPs).
 - **SGD with Momentum** is often preferred in Computer Vision models (ResNets) because it can achieve better generalization when tuned.
-

Module 8: Loss Functions

- **Loss Function**: Evaluates error on a single training instance.
- **Cost Function**: The average of the loss function across all training instances.
- **Common Loss Functions**
 - **Binary Cross Entropy (BCE)**: For binary classification:

$$\text{Loss} = -[y \log(p) + (1 - y) \log(1 - p)]$$

- **Categorical Cross Entropy**: For multi-class classification:

$$\text{Loss} = - \sum_{i=1}^C y_i \log(p_i)$$

- **Mean Squared Error (MSE)**: For regression, penalizing larger errors heavily.
 - **Mean Absolute Error (MAE)**: For regression, robust to outliers.
 - **Huber Loss**: Behave as MSE for small errors and MAE for large errors, combining the benefits of both.
 - **Dice Loss**: Used in image segmentation. Measures the overlap (Intersection over Union) between prediction and ground truth.
 - **Focal Loss**: Used in object detection. Downweights easy-to-classify samples, forcing the model to focus on hard-to-classify objects in imbalanced datasets.
-

Module 9: Training Concepts

- **Epoch**: One complete pass of the entire training dataset through the neural network.
- **Batch Size**: The number of training samples processed before the network updates its internal weights.
- **Iteration (Steps Per Epoch)**: The number of batches needed to complete one epoch:

$$\text{Steps} = \frac{\text{Total Training Samples}}{\text{Batch Size}}$$

- **Early Stopping:** A validation technique that halts training when validation loss stops improving for a specified number of epochs (patience), preventing overfitting.
 - **Checkpointing:** Periodically saving model weights during training so that training can be resumed from the last saved state in case of hardware failures.
-

Module 10: Overfitting

- **Overfitting:** High training accuracy but low validation/test accuracy. High Variance, Low Bias.
 - **Underfitting:** Low training and validation accuracy. High Bias, Low Variance.
 - **Data Augmentation:** Artificially increasing dataset size by applying random transformations (e.g., rotations, flips, brightness adjustments on images) to prevent overfitting.
-

Module 11: Regularization

- **L1/L2 Regularization:** Adds weight magnitude penalties to the loss function to prevent weights from growing too large.
 - **Dropout:** Randomly deactivates a fraction of neurons (e.g., 20-50%) during each training step. Prevents neurons from co-adapting and forces the network to learn redundant representations.
 - **Batch Normalization:** Normalizes inputs of each layer across the mini-batch to have a mean of 0 and variance of 1. Speeds up training and acts as a mild regularizer.
 - **Layer Normalization:** Normalizes inputs across features within a single sample. Essential for sequential architectures like RNNs and Transformers.
 - **Weight Decay:** The implementation equivalent of L2 regularization, subtracting a small fraction of the weight at each parameter update step.
-

Module 12: CNN (Convolutional Neural Network)

Designed for processing grid-like spatial data (images) while minimizing parameter count.

- **Why CNN?** Traditional dense networks fail on images because flat input dimensions explode parameter counts, discarding spatial relationships. CNNs preserve spatial relationships using weight sharing and local receptive fields.
- **Components**
 - **Convolution Layer:** Applies sliding kernels (filters) over inputs to produce feature maps.

- **Kernel/Filter:** A small matrix (e.g., 3×3) containing learnable weights that detect features like edges or textures.
 - **Stride:** The step size of the kernel as it slides across the image.
 - **Padding:** Adding pixels (usually zeros) to the border of the image. **Valid Padding** means no padding (output shrinks). **Same Padding** pads borders so output dimensions match input dimensions.
 - **Pooling Layer:** Downsamples feature maps to reduce spatial dimensions. Includes **Max Pooling** (extracts the maximum value) and **Average Pooling** (extracts the average value).
 - **Flatten Layer:** Unrolls 2D feature maps into a 1D vector.
 - **Fully Connected (Dense) Layer:** Feeds the flattened vector to dense layers for final classification.
-

Module 13: CNN Architectures

- **LeNet-5 (1998):** Early handwriting digit recognizer containing basic convolutional and average pooling layers.
 - **AlexNet (2012):** Won the ImageNet challenge. Introduced deep stacked convolutions, ReLU, Dropout, and GPU acceleration.
 - **VGG16 / VGG19:** Used small 3×3 convolutions stacked deeply, showing that depth is a key factor in network representation.
 - **GoogLeNet (Inception):** Introduced Inception modules, applying parallel convolutions of different sizes (1×1 , 3×3 , 5×5) and pooling together to capture multi-scale features.
 - **ResNet:** Introduced **Residual blocks** with skip connections ($F(x) + x$) to solve the vanishing gradient problem, enabling the training of extremely deep networks (e.g., 152 layers).
 - **DenseNet:** Connects every layer to every other layer in a feed-forward fashion, improving feature propagation and reducing parameter count.
 - **EfficientNet:** Scales network depth, width, and resolution uniformly using a compound scaling coefficient, achieving state-of-the-art accuracy with high efficiency.
 - **MobileNet:** Employs **depthwise separable convolutions** to build lightweight models optimized for mobile and edge devices.
-

Module 14: Transfer Learning

- **Transfer Learning** Taking a model trained on a large dataset (e.g., ImageNet) and adapting it to a smaller, specific target dataset.
- **Strategies**
 - **Feature Extraction:** Freezing the pre-trained layers (keeping weights static) and training only the new custom classification head on the target dataset.

- **Fine-Tuning:** Unfreezing some or all of the pre-trained layers and training them on the target dataset with a very low learning rate.
 - **Frozen Layers:** Layers whose weights are locked during training to preserve pre-trained features.
-

Module 15: RNN (Recurrent Neural Network)

Designed for processing sequential or time-series data.

- **Why RNN?** Traditional networks assume inputs are independent. RNNs process inputs sequentially, passing an internal **Hidden State** (h_t) from step to step to act as memory:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b)$$

- **Limitations:** Struggle to retain long-term dependencies because backpropagating gradients over many time steps (Backpropagation Through Time) causes gradients to vanish or explode.
-

Module 16: LSTM (Long Short-Term Memory)

An advanced RNN architecture designed to preserve long-term dependencies, resolving the vanishing gradient problem.

- **Architecture Components** Uses a **Cell State** (C_t) as a constant error carousel, controlled by three gates:
 - **Forget Gate:** Decides what information to discard from the cell state. Uses Sigmoid.
 - **Input Gate:** Decides what new information to add to the cell state. Uses Sigmoid and Tanh.
 - **Output Gate:** Decides what information from the cell state to output as the hidden state.
-

Module 17: GRU (Gated Recurrent Unit)

A simplified variation of LSTM.

- **Key Differences**
 - Combines the cell state and hidden state into a single hidden state.
 - Uses only two gates: **Update Gate** (controls forget/input decisions) and **Reset Gate** (controls how much past memory to forget).

- **GRU vs LSTM** GRU contains fewer parameters, is less prone to overfitting on small datasets, and trains faster. LSTM is more expressive and can perform better on complex, long-sequence datasets.
-

Module 18: Attention

- **What is Attention?** A mechanism that allows models to focus on specific parts of the input sequence when generating output, regardless of their distance in the sequence.
 - **Why Attention?** Traditional seq2seq RNNs compress entire input sequences into a single fixed-length context vector, creating a bottleneck for long sentences. Attention resolves this by calculating alignment scores between output and input tokens.
 - **Self-Attention:** Computes attention weights by comparing each word in a sequence with all other words in the same sequence (Query, Key, Value calculations).
 - **Multi-Head Attention:** Runs multiple self-attention operations in parallel, allowing the model to attend to information from different representation subspaces simultaneously.
-

Module 19: Transformers

An architecture introduced in the paper "Attention Is All You Need" (2017) that discards recurrence and convolution entirely, relying solely on self-attention mechanisms. This allows inputs to be processed in parallel, accelerating training.

- **Architecture Components**
 - **Encoder:** Processes the input sequence in parallel, generating contextual embeddings.
 - **Decoder:** Generates the output sequence autoregressively, using self-attention and encoder-decoder cross-attention.
 - **Positional Encoding:** Adds sinusoidal vectors to input embeddings to provide token order information, as Transformers lack recurrence.
 - **Self-Attention & Multi-Head Attention:** (See Module 18).
 - **Feed-Forward Networks (FFN):** Applied to each position independently after the attention layers.
-

Module 20: Large Language Models (LLMs)

- **What is LLM?** A Transformer-based model containing billions of parameters, trained on massive web-scale text corpora.
- **Architectures**

- **GPT (Generative Pre-trained Transformer)**: Decoder-only architecture trained to predict the next token (Autoregressive).
 - **BERT**: Encoder-only architecture trained using Masked Language Modeling to understand bidirectional context.
 - **T5**: Encoder-Decoder architecture that treats all NLP tasks as text-to-text transformations.
 - **LLaMA**: Meta's open-weights autoregressive decoder-only model series.
 - **Parameters & Concepts**
 - **Tokenization**: Splitting text into sub-word tokens (using algorithms like Byte-Pair Encoding).
 - **Embedding**: Mapping token IDs to continuous high-dimensional vectors.
 - **Context Window**: The maximum number of tokens the model can process in a single prompt execution.
 - **Temperature**: A parameter controlling randomness during token selection (low temp: deterministic; high temp: creative/random).
-

Module 21: Autoencoders

- **What is Autoencoder?** An unsupervised network trained to reconstruct its inputs.
 - **Encoder**: Compresses input data into a lower-dimensional bottleneck layer (latent space).
 - **Decoder**: Reconstructs the original input from this latent space representation.
 - **Reconstruction Error**: The loss function (e.g. MSE) measuring differences between input and reconstructed output.
 - **Variational Autoencoder (VAE)** A generative autoencoder. Instead of mapping inputs to a fixed point in latent space, the encoder outputs the mean (μ) and variance (σ) of a probability distribution. The decoder samples from this distribution. Regularized using **KL-Divergence** loss.
-

Module 22: GAN (Generative Adversarial Network)

A generative framework containing two networks competing in a minimax game:

- **Generator**: Creates realistic fake data from random noise vectors, aiming to fool the discriminator.
- **Discriminator**: Classifies data as real (from the training set) or fake (from the generator), aiming to detect generator fakes.

- **GAN Workflow** [Noise] → [Generator] → [Fake Data] \ → [Discriminator] → [Real/Fake Class] [Real Data] / The networks are trained alternately. The Generator tries to maximize the likelihood of the Discriminator making a mistake.
 - **Applications:** Deepfakes, high-resolution upscaling (SRGAN), text-to-image synthesis, and data augmentation.
-

Module 23: Object Detection

Locates and classifies objects within an image.

- **Algorithms**
 - **YOLO (You Only Look Once):** A single-stage detector. Treats object detection as a single regression problem, predicting bounding boxes and class probabilities in a single pass. Extremely fast.
 - **SSD (Single Shot MultiBox):** Single-stage detector that uses multi-scale feature maps to detect objects of various sizes.
 - **Faster R-CNN:** Two-stage detector. A Region Proposal Network (RPN) suggests candidate regions, which are then classified and refined. High accuracy, slower.
 - **Concepts**
 - **Bounding Box:** Coordinates defining the location of an object (x, y, w, h) .
 - **IoU (Intersection over Union):** Evaluates overlap between predicted and ground-truth bounding boxes.
 - **Non-Maximum Suppression (NMS):** Eliminates redundant, overlapping bounding boxes, retaining only the box with the highest confidence score.
-

Module 24: Image Segmentation

- **Semantic Segmentation:** Classifies every pixel in an image into a class (e.g., Road vs. Sky). Does not distinguish between different instances of the same class.
 - **U-Net:** An encoder-decoder architecture with skip connections, standard for medical semantic segmentation.
 - **Instance Segmentation:** Classifies pixels and distinguishes between different objects of the same class.
 - **Mask R-CNN:** Extends Faster R-CNN by adding a branch that outputs pixel-level segmentation masks for each Region of Interest.
-

Module 25: NLP (Natural Language Processing)

- **Text Cleaning:** Tokenization, Stemming (crude word chopping), Lemmatization (using vocabulary to find dictionary base form), and Stop Word removal.
 - **Word Embeddings:** Vector representations mapping words to high-dimensional space where semantic relationships are preserved.
 - *Word2Vec:* Uses Continuous Bag of Words (CBOW) or Skip-gram architectures.
 - *GloVe (Global Vectors):* Learns embeddings by factoring global word co-occurrence matrices.
 - *FastText:* Treats words as bags of character n-grams, enabling it to generate embeddings for out-of-vocabulary words.
-

Module 26: Deployment

- **Model Formats:** Saving models as `.h5` (Keras), PyTorch `.pt`, or TensorFlow `SavedModel`.
 - **ONNX (Open Neural Network Exchange):** An open format built to represent machine learning models, allowing interoperability between PyTorch, TensorFlow, and other frameworks.
 - **TorchScript:** A way to create serializable and optimizable models from PyTorch code, allowing them to run in C++ environments.
 - **TensorRT:** NVIDIA's high-performance deep learning inference optimizer and runtime for GPUs.
-

Module 27: MLOps

- **Model Drift:** Degradation of model performance in production due to changes in input distributions over time.
 - **Experiment Tracking:** Logging training metrics, parameters, and models using tools like **MLflow**.
 - **GPU vs CPU:** CPUs process tasks sequentially on a few cores. GPUs parallelize math operations across thousands of cores, making them ideal for matrix operations in deep learning.
 - **CUDA:** NVIDIA's parallel computing platform that allows developers to run deep learning computations directly on GPUs.
 - **Mixed Precision:** Training using a mix of 16-bit (FP16) and 32-bit (FP32) floating-point numbers. Speeds up GPU training and saves memory without sacrificing accuracy.
 - **Model Quantization:** Reducing weight precision (e.g. FP32 → INT8) to speed up inference and run models on edge devices with limited resources.
-

★ Top 40 Most Frequently Asked Deep Learning Questions (Accenture)

1. **What is Deep Learning?** A subset of ML using deep neural networks with multiple hidden layers to automatically extract features from raw data.
2. **AI vs ML vs DL?** AI is the concept of intelligent machines; ML is a data-driven subset of AI; DL is a subset of ML using deep neural networks.
3. **What is an ANN?** Artificial Neural Network: A layered structure of interconnected nodes modeling input-output mappings.
4. **What is a Perceptron?** The simplest single-layer neural network unit:
 $y = \text{Activation}(Wx + b)$.
5. **What is the XOR problem?** A classic demonstration that a single-layer perceptron cannot classify non-linearly separable data, requiring multi-layer networks.
6. **Why use Activation Functions?** To introduce non-linearity into the network. Without them, deep layers behave like a single linear regression model.
7. **Sigmoid vs ReLU?** Sigmoid maps values to range $(0, 1)$ but saturates, causing vanishing gradients. ReLU ($\max(0, x)$) is faster and does not saturate for positive inputs.
8. **What is Softmax?** An activation function that converts a vector of numbers into a probability distribution summing to 1. Used in multi-class classification.
9. **What is Forward Propagation?** The process of passing inputs through the network layers to calculate the final output and loss.
10. **What is Backpropagation?** The algorithm used to calculate the gradient of the loss function with respect to weights using the chain rule, moving backward through the network.
11. **What is Gradient Descent?** An optimization algorithm that updates weights in the direction of the negative gradient to minimize the loss function.
12. **What is the Learning Rate?** A hyperparameter controlling the step size taken during optimization updates.
13. **Adam vs SGD?** Adam uses adaptive learning rates for each parameter, converging faster. SGD uses a uniform learning rate, which is slower but can generalize better.
14. **What is a Loss Function?** A mathematical function measuring the error of a model's prediction on a single training sample.
15. **Epoch vs Batch vs Iteration?**
 - **Epoch:** One full pass through the entire dataset.
 - **Batch:** A subset of samples processed before updating weights.
 - **Iteration:** The execution of one batch update.
16. **How do you detect Overfitting?** High training accuracy combined with low validation/test accuracy.
17. **What is Dropout?** A regularization technique that randomly deactivates a percentage of neurons during training to prevent co-adaptation.
18. **What is Batch Normalization?** Normalizing the inputs of each layer across the mini-batch, speeding up training and acting as a mild regularizer.

19. **What is a CNN?** Convolutional Neural Network: A network optimized for processing grid-like spatial data (images) using filters.
20. **What is a Convolutional Layer?** A layer that applies sliding kernels over inputs to extract spatial feature maps.
21. **What is a Pooling Layer?** A layer used to downsample feature maps, reducing spatial dimensions and parameter count.
22. **What is Transfer Learning?** Adapting a model trained on a large dataset (e.g. ImageNet) to a new, smaller target dataset.
23. **What is an RNN?** Recurrent Neural Network: A network designed for sequential data that passes a hidden state memory from step to step.
24. **What is an LSTM?** Long Short-Term Memory: An advanced RNN that resolves the vanishing gradient problem using a cell state and three gates.
25. **What is a GRU?** Gated Recurrent Unit: A simplified LSTM that uses two gates and combines the cell and hidden states.
26. **Vanishing vs Exploding Gradients?**
 - **Vanishing:** Gradients shrink close to 0 during backpropagation, preventing weight updates in early layers.
 - **Exploding:** Gradients grow exponentially, causing weights to oscillate and diverge.
27. **What is the Attention Mechanism?** An alignment mechanism allowing models to focus on specific parts of the input sequence when generating outputs.
28. **What is a Transformer?** An architecture that replaces recurrence with self-attention, allowing inputs to be processed in parallel.
29. **What is Self-Attention?** An attention mechanism that compares each word in a sequence with all other words in the same sequence to calculate context.
30. **BERT vs GPT?**
 - **BERT:** Encoder-only Transformer trained bidirectionally to understand context.
 - **GPT:** Decoder-only Transformer trained autoregressively to predict the next token.
31. **What is an LLM?** Large Language Model: A Transformer-based model containing billions of parameters trained on web-scale text.
32. **What is an Autoencoder?** An unsupervised network that compresses inputs into a lower-dimensional bottleneck and reconstructs them.
33. **What is a GAN?** Generative Adversarial Network: A system where a Generator and a Discriminator compete in a minimax game to generate realistic data.
34. **What is YOLO?** You Only Look Once: A fast single-stage object detector that predicts bounding boxes and class probabilities in a single pass.
35. **What is a U-Net?** An encoder-decoder architecture with skip connections, standard for semantic image segmentation.
36. **GPU vs CPU?** CPUs process tasks sequentially on a few cores. GPUs parallelize matrix operations across thousands of cores, making them ideal for deep learning.

37. **What is CUDA?** NVIDIA's parallel computing platform that allows developers to run deep learning computations directly on GPUs.
38. **What is Model Quantization?** Reducing weight precision (e.g. FP32 $\rightarrow$ INT8) to speed up inference and run models on edge devices.
39. **What is ONNX?** Open Neural Network Exchange: An open format that allows models to be shared across different frameworks.
40. **How do you handle Model Drift?** Monitor production input data distributions and retrain the model on new labeled data when drift is detected.



Module Summary & Key Takeaways

Theory Focus: Backpropagation, Convolutional/Recurrent Networks, Transformers, and MLOps.

This module details modern deep learning networks and model parameter optimization. Core concepts to retain include:

- **Backpropagation & Optimization:** Feedforward networks calculate loss values, propagating gradients backward through layers using the Chain Rule. Weight parameters are updated via gradient descent variants (SGD, RMSprop, Adam) to minimize error metrics.
- **Architectural Patterns:**
 - **CNNs** use convolutional kernels and max-pooling to extract spatial hierarchical patterns, making them suitable for images.
 - **RNNs / LSTMs / GRUs** capture sequential dependencies in text and time-series, mitigating vanishing gradients through gates that regulate memory states.
 - **Transformers** replace recurrence with self-attention mechanisms, processing sequences in parallel to compute global context dependencies, enabling modern LLMs.
- **Generalization & Deployment:** Overfitting is managed via dropout, early stopping, and batch normalization, preparing models for production serving via MLOps deployment cycles.